HS Harz FuSa, Friedrichstraße 57

Airbag ECU

Messung und Verarbeitung der Sensoreingänge zur Auslösung des Airbags
Measurement and Processing of Sensor Signals to Process an Airbag Operation

Phase 12: Software Implementation

Version: 1.00

File : Airbag_ECU_Phase_12
Version : 1.00
Date : 2026/14/06
Editor : Shruti Pawar

Contents

1 Meetings within the project 3

2 Unresolved topics 4

3 Glossar 5

4 System Level Requirements ..... Error! Bookmark not defined.

4.1 Timing Requirements Operating Environment..... **Error! Bookmark not defined.**

4.2 Safety-Relevant Functions ASIL-D Compliance Mandates **Error! Bookmark not defined.**

4.3 Fault Tolerance Requirements Safe State Trigger Conditions **Error! Bookmark not defined.**

4.4 Compliance Considerations ..... **Error! Bookmark not defined.**

5 Figure and table listing 6

6 List of dependent documents 11

7 Literature 12

8 Version history 13

1 Meetings within the project

Topic	Time	Room	Participants
Item Definition	15:00	Zoom	ALL
Bericht:	*none*		

2 Unresolved topics

No unresolved topics

3 Glossar

Tabelle 1: Glossar

Begriff / Term	Erklaerung / Explanation
ACU	Airbag Control Unit.
ADC	Analog to digital converter value read from an analog sensor input.
ASIL	Automotive Safety Integrity Level used by ISO 26262 for risk classification.
CAN	Controller Area Network, a vehicle communication bus.
ECU	Electronic Control Unit.
EUC	Equipment under Control.
FuSa	Functional Safety.
HW	Hardware.
QM	Quality Management level below ASIL.
SIL	Safety Integrity Level.
SW	Software.

4 Software Implementation

This section documents the full Arduino code implementation for the Airbag ECU prototype, code explanation, and ASIL-divided problem/solution analysis. (Source: FuSa Report, Section 9 / ASIL Code Division V2, Section 5.4)

4.1 Arduino Code

The following is the complete ASIL-divided Arduino prototype code. (Source: ASIL Code Division V2, Section 5.4)

const int accSensor = A0; const int gyroSensor = A1; const int pressureSensor = A2;

const int babySeatPin = 2; const int driverAirbag = 8; const int passengerAirbag = 9; const int warningLamp = 13;

const int ACC_THRESHOLD = 700; const int GYRO_THRESHOLD = 600; const int PRESSURE_THRESHOLD = 650; const int SENSOR_MAX_VALID = 950;

void setup() { Serial.begin(9600); pinMode(driverAirbag, OUTPUT); pinMode(passengerAirbag, OUTPUT); pinMode(warningLamp, OUTPUT); pinMode(babySeatPin, INPUT_PULLUP); digitalWrite(driverAirbag, LOW); digitalWrite(passengerAirbag, LOW); digitalWrite(warningLamp, LOW); Serial.println("Airbag ECU ASIL-divided prototype started"); }

void loop() { readInputsQM(); systemFault = checkSensorValidity_ASIL_B(); if (systemFault) { enterSafeState_ASIL_B(); } else { evaluateDeploymentDecision_ASIL_D(); applyPassengerSuppression_ASIL_C(); setOutputs_ASIL_D(); if (!verifyOutputState_ASIL_B()) { enterSafeState_ASIL_B(); } } sendDiagnosticFrame_ASIL_A(); printDebugInformation_QM(); delay(1000); }

4.2 Code Explanation

(Source: FuSa Report, Section 10)

setup(): Initializes the ECU prototype which sets the sensor pins, output pins and warning lamp and serial communication.

loop(): Works as the cyclic ECU task which continuously reads the sensor inputs, evaluates the crash logic and checks the safety conditions and sends diagnostic information to make the decision about the deployment.

checkSensorValidity_ASIL_B(): Checks whether the sensor values are inside valid ADC range. If invalid data is detected then the software goes into safe state.

evaluateDeploymentDecision_ASIL_D(): Converts the raw sensor readings into crash condition using ACC_THRESHOLD, GYRO_THRESHOLD, PRESSURE_THRESHOLD.

applyPassengerSuppression_ASIL_C(): Even when a valid crash is detected, passengerCommand is forced false when babySeatDetected is true.

setOutputs_ASIL_D(): Activates the outputs. The driver airbag is activated during valid crash. The passenger airbag is activated only when the baby seat is not detected.

verifyOutputState_ASIL_B(): Performs output readback verification.

enterSafeState_ASIL_B(): Disables deployment output and turns on the warning lamp.

sendDiagnosticFrame_ASIL_A(): Creates an 8-byte diagnostic frame representing CAN communication.

4.3 Simulation Evidence

The following figures show the prototype running in simulation with actual serial monitor output for each key scenario:

Fig 01: Normal State ACC=349, GYRO=299, PRESSURE=345, BABY=NO. SCENARIO: NORMAL / NO VALID CRASH. Driver Airbag OFF, Passenger Airbag OFF. No crash threshold exceeded.

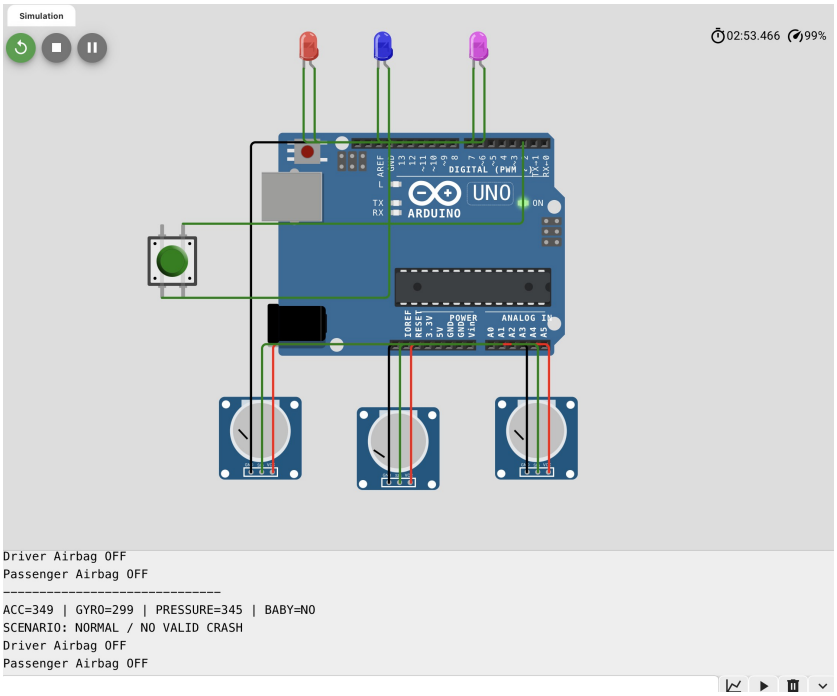

Abbildung 1 Normal State No Valid Crash

Fig 02: Signal with defective Airbag — GYRO=1023 causes WARNING: SENSOR VALUE TOO HIGH. Driver Airbag OFF, Passenger Airbag OFF, warning lamp ON. Safe state activated.

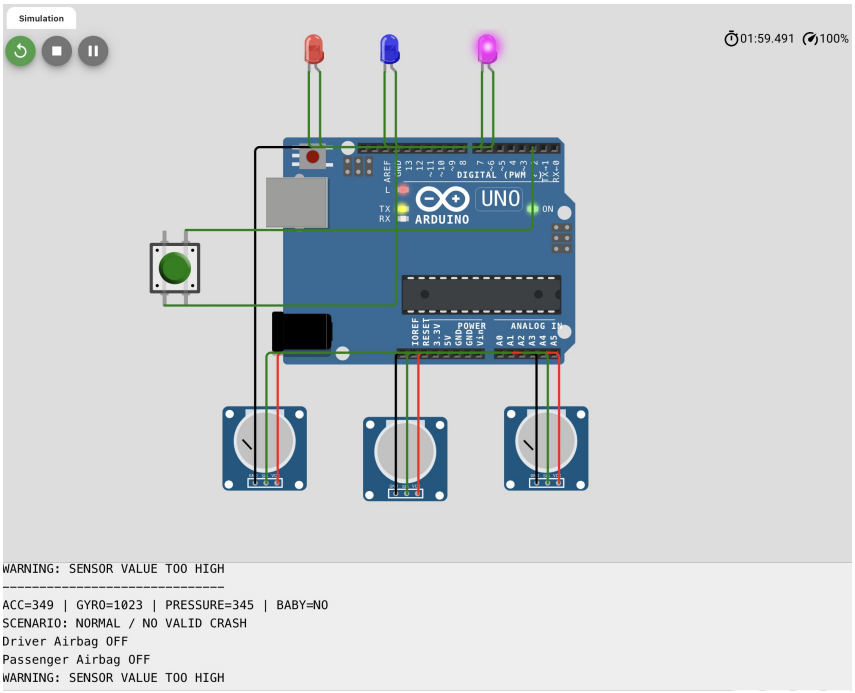

Abbildung 2 Signal with Defective Airbag Safe State Activated

Fig 03: Baby Seat Detected ACC=807, GYRO=633, PRESSURE=227, BABY=YES. SCENARIO: VALID CRASH + BABY SEAT. Driver Airbag ON, Passenger Airbag SUPPRESSED.

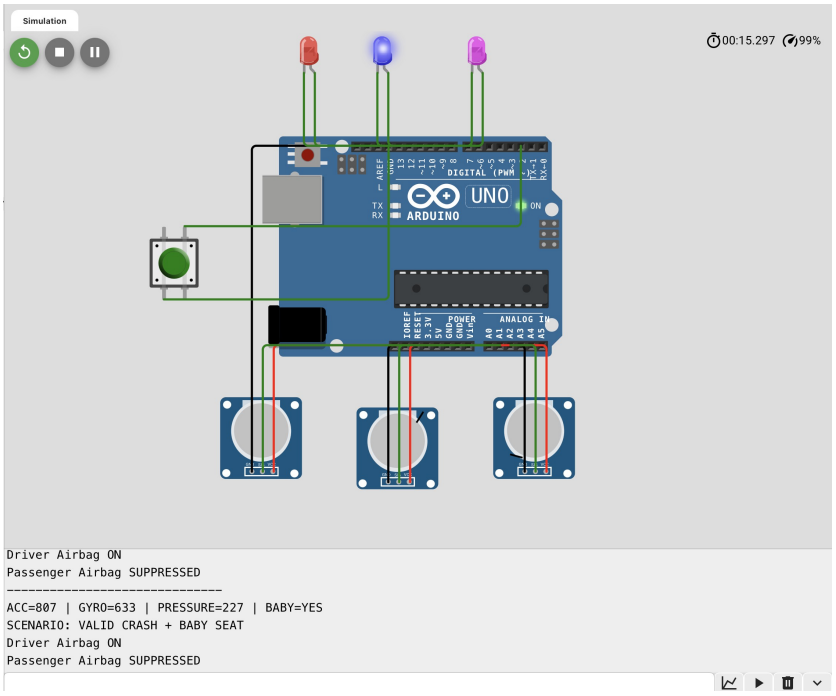

Abbildung 3 Baby Seat Detected Valid Crash, Passenger Airbag SUPPRESSED

Fig 04: Frontal and Side Accident ACC=705, GYRO=716, PRESSURE=242, BABY=NO. SCENARIO: VALID CRASH. Driver Airbag ON, Passenger Airbag ON.

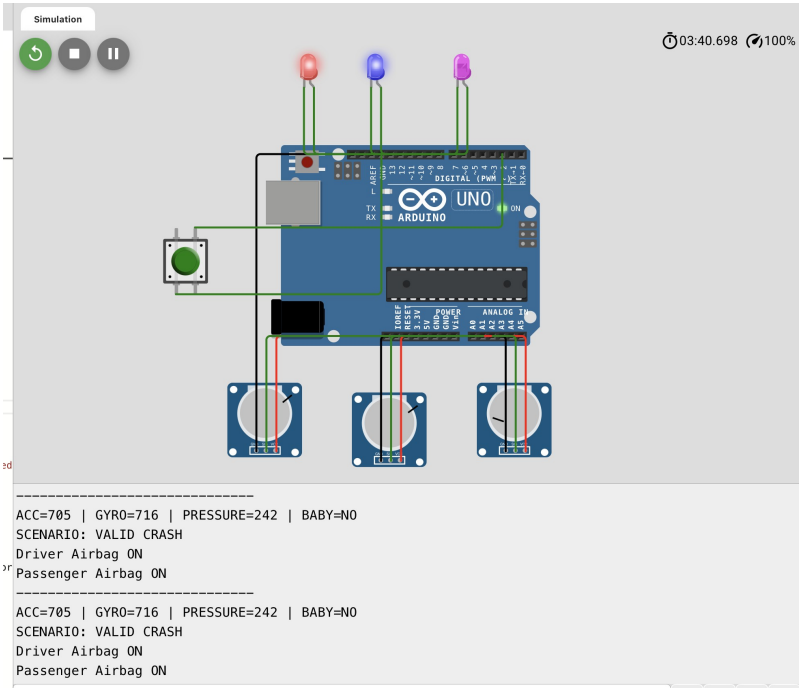

Abbildung 4 Frontal and Side Accident Valid Crash, Driver and Passenger Airbag ON

4.4 Problems and Solutions by ASIL Category

Tabelle 2 Problems and Solutions by ASIL Category

ASIL	Code Area	Problem	Solution / Safety Measure	Reference
ASIL D	evaluateDeploymentDecision_ASIL_D(), setOutputs_ASIL_D()	A single high sensor value could cause unintended deployment. A missed valid crash could also prevent required protection.	Use a valid-crash decision with crash detection plus a safing condition and no system fault. The prototype uses acceleration/pressure thresholds together with safing validation before deployment.	SWR-01 to SWR-05 [P1]; ISO 26262 [1]
ASIL C	applyPassengerSuppression_ASIL_C()	Passenger airbag output may be unsafe when a baby seat is detected.	Use a separate passenger suppression interlock. passengerCommand is forced false when babySeatDetected is true.	SWR-04 and SWR-06 [P1]
ASIL B	checkSensorValidity_ASIL_B(), verifyOutputState_ASIL_B(), enterSafeState_ASIL_B()	Invalid sensor values or output mismatch can make the system unreliable.	Detect invalid sensor values, compare output readback against commanded output, and enter safe state.	Safe-state logic and ST-06/ST-07 test plan [P1]
ASIL A	sendDiagnosticFrame_ASIL_A()	Crash/fault information may not be visible to other ECUs or to the tester.	Transmit/print an 8-byte diagnostic frame using CAN-style fields: status, sensor values, passenger state, deployment status, fault code and checksum.	CAN frame design [P1]; Arduino CAN [4]
QM	readInputsQM(), printDebugInformation_QM()	Debug text can help testing but must not be considered a safety mechanism.	Keep serial logging separate from deployment logic. It may be used for test observation only.	Test routine plan [P1]

5 Figure and table listing

Tabelle 1: Glossar 5
Tabelle 2 Problems and Solutions by ASIL Category 9

Abbildung 1 Normal State No Valid Crash 7
Abbildung 2 Signal with Defective Airbag Safe State Activated 7
Abbildung 3 Baby Seat Detected Valid Crash, Passenger Airbag SUPPRESSED 8
Abbildung 4 Frontal and Side Accident Valid Crash, Driver and Passenger Airbag ON 8

6 List of dependent documents

No.	Dokument name	Version	Date	File name
1	Arduino Coding and Logic Implementation for Airbag ECU System	V1	02.06.2026	FuSa Report_Adruino.docx
2	Airbag ECU ASIL Code Division	V02	02.06.2026	Airbag_ECU_ASIL_Code_Division_V02.docx

7 Literature

[1] ISO, ISO 26262-1:2011 Road vehicles - Functional safety. ISO 26262 addresses hazards caused by malfunctioning behaviour of E/E safety-related systems in road vehicles.

[2] Synopsys, What is ASIL (Automotive Safety Integrity Level)? ASIL is a risk classification system under ISO 26262; ASIL A is lowest and ASIL D is highest.

[3] Arduino Documentation, Due. Arduino Due is based on a 32-bit ARM core and provides 54 digital I/O pins, 12 analog inputs and CAN capability.

[4] Arduino Documentation, Controller Area Network (CAN) Bus. CAN is a robust vehicle bus standard for communication between microcontrollers/devices without a host computer.

[P1] Atashi Saha, Arduino Coding and Logic Implementation for Airbag ECU System, uploaded project report, 2026.

[P2] HS Harz FuSa, Airbag ECU Portfolio HS Harz FuSa - Concept template, version 1.00, 2026/04/29

8 Version history

Document name	date	Change log
Airbag_ECU_Phase_12	14.06.2026	v1.00 Initial version. Software Implementation compiled from FuSa Report Sections 9, 10 and ASIL Code Division V2 Sections 5.4, 5.5.